

Trip Planner

LLM-агент персонального планирования путешествий

КУРС NFACTORIAL — LLM STACK

Проблема

Туристическое планирование «руками» — это:

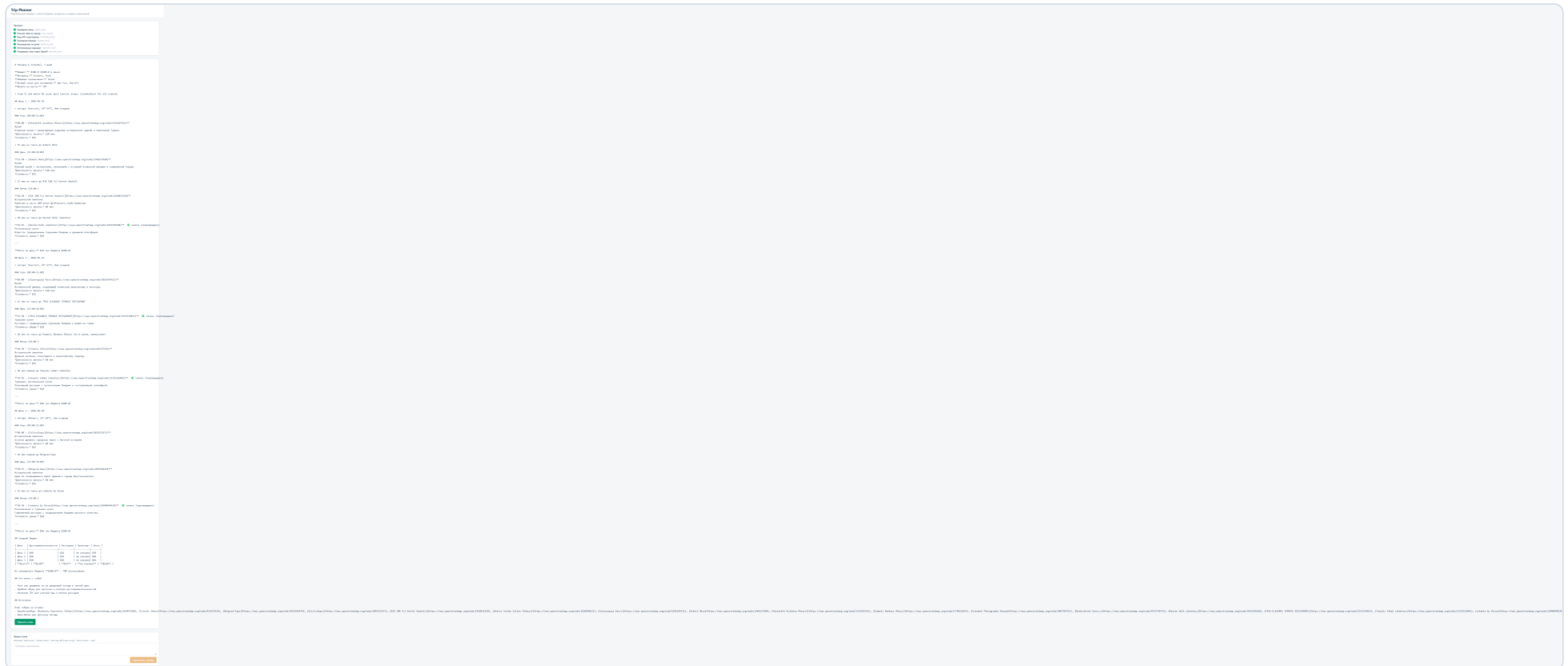
- **Часы скроллинга** TripAdvisor / Reddit / блогов в попытке собрать день по часам
- **Бюджет** не отслеживается — реалистичная стоимость становится понятной постфактум
- **Dietary restrictions** (halal, vegan, gluten-free) проверяются вручную по каждому ресторану
- **Погода и логистика** не сшиваются с маршрутом
- **Правки:** «убери музеи, добавь халяль» — никакой инструмент не делает это точно

Решение

Веб-приложение, которое за **~1-4 минуты** собирает персонализированный day-by-day план:

1. Пользователь задаёт **город, дни, бюджет, интересы, dietary**; опционально — фото места.
2. LLM-агент через **LangGraph + 3 MCP-сервера + RAG** собирает план: реальные POI из OSM, рестораны с halal-маркерами, погода, оптимизированный маршрут.
3. Пользователь пишет правки в свободной форме — точечный патч, не регенерация.
4. **Accept → PDF.**

User flow — форма



Vite + React 19 + Tailwind + shadcn/ui. Валидация zod, SSE-прогресс по нодам графа.

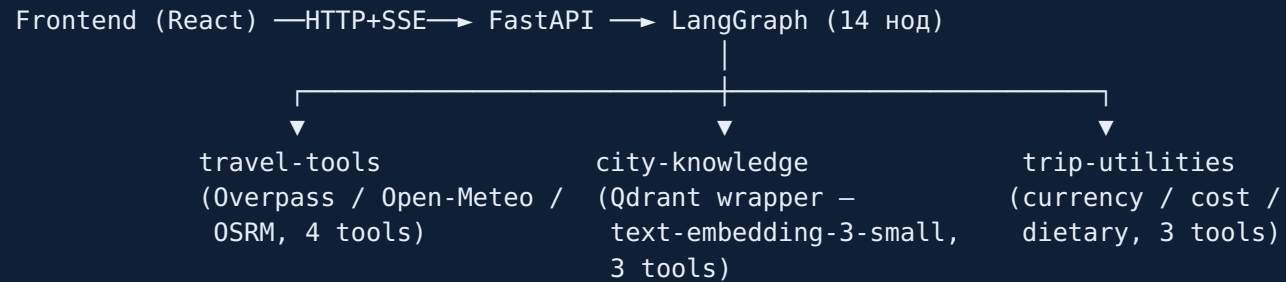
User flow — готовый план

The screenshot displays the 'Trip Planner' application interface. At the top, it shows the trip title 'Поездка в Истанбул, 3 дня' and the dates '10-12-2023' to '13-12-2023'. Below this, the itinerary is divided into three days. Each day's plan includes a title, a brief description, and a list of activities with their respective costs. For example, Day 1 includes 'Визит в Стамбул' and 'Посещение музея Галатийской башни'. At the bottom, there is a table titled 'Сводный бюджет' (Summary Budget) showing the total cost for each day and the overall total.

День	Активность	Стоимость	Всего
1	Визит в Стамбул	10	10
2	Посещение музея Галатийской башни	15	25
3	Посещение музея Галатийской башни	15	40
Итого			40

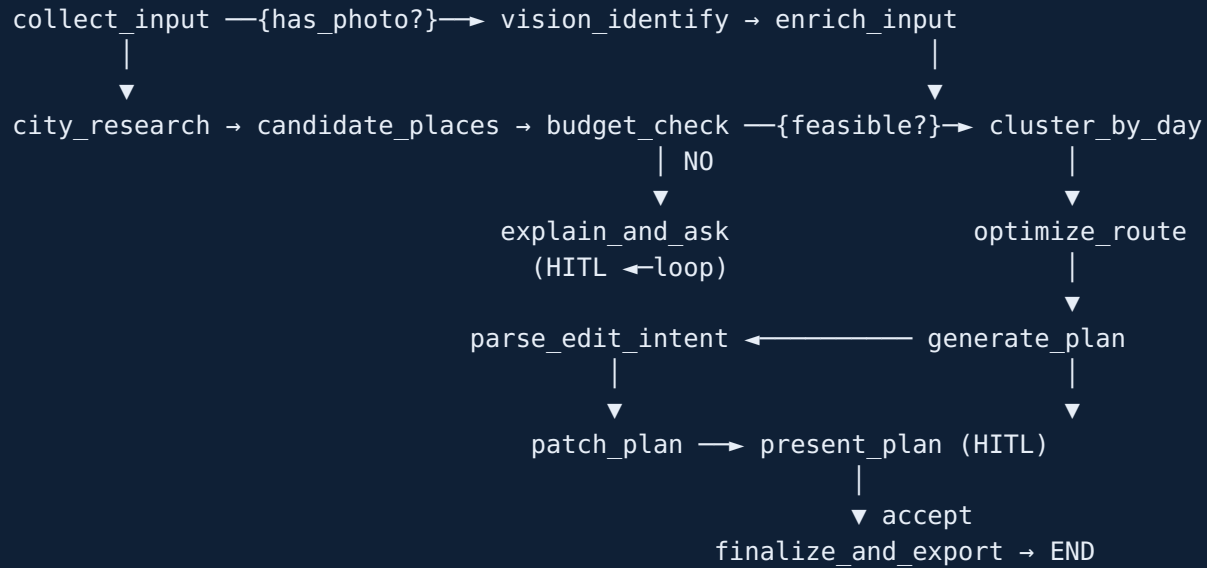
Погода (Open-Meteo), halal-маркеры 🕌, переходы между точками, OSM-ссылки на каждое место, таблица сводного бюджета, источники (Wikivoyage + OSM).

Архитектура — high-level



Один docker-compose: qdrant + backend + frontend. MCP-серверы — stdio-subprocess'ы backend'a (не отдельные compose-сервисы). LangSmith трассирует всё автоматически.

LangGraph — 14 нод, 3 ветвления, 2 цикла, 2 HITL



`AsyncSqliteSaver` checkpointing. `EVAL_MODE=true` обходит HITL для автопрогона evals.

MCP server #1 — travel-tools

Назначение: доступ к внешнему миру через бесплатные публичные API. Никакой LLM-логики.

Tool	Источник	Возвращает
<code>find_places</code>	Overpass (OSM)	список <code>Place</code> по категориям (museum / park / historical / religious / nightlife / shopping)
<code>find_restaurants</code>	Overpass <code>diet:*</code> теги	<code>Restaurant</code> с <code>dietary_confidence</code>
<code>get_weather_forecast</code>	Open-Meteo	дневной прогноз, до 16 дней
<code>compute_route</code>	OSRM	<code>RouteResult</code> (длина/время) для walk/transit/drive

Resilience: `_overpass_query` крутит 5 попыток через 3 зеркала Overpass с экспоненциальным backoff — main endpoint регулярно отдаёт 504.

MCP server #2 — city-knowledge (RAG wrapper)

Назначение: агент через MCP обращается к собственному RAG-индексу — не только внешние API.

Tool	Возвращает
<code>search_city_guide(city, query, k, section?)</code>	top-k <code>GuideChunk</code> из Wikivoyage
<code>get_city_overview(city)</code>	<code>CityOverview</code> — валюта, языки, безопасность, транспорт, лучший сезон
<code>list_indexed_cities()</code>	какие города проиндексированы (graceful fallback)

Stack: `text-embedding-3-small` (1536-dim) → Qdrant `query_points(query=vector, query_filter=Filter(city=..., kind=...))`. Одна коллекция на все города, фильтр по metadata-полю `city`.

MCP server #3 — trip-utilities

Назначение: доменные утилиты — без сети, чистая логика.

Tool	Что делает
<code>convert_currency</code>	конвертация через frankfurter.app
<code>estimate_plan_cost</code>	агрегирует стоимость по дням → <code>CostBreakdown</code>
<code>validate_dietary_match</code>	эвристика по cuisine + OSM <code>diet:*</code> тегам → <code>accommodates</code> / <code>partial</code> / <code>does_not_accommodate</code>

Skill — `itinerary-formatter`

`skill/itinerary-formatter/SKILL.md` — единый формат для UI и PDF.

Вызывается в двух нодах: `generate_plan` и `finalize_and_export`.

Hard rules:

1. Никогда не упоминать место, которого нет в `state.candidate_places` / `candidate_restaurants` / `city_context`.
2. Каждое место — ссылка на OSM или Wikivoyage.
3. Цены/часы — только из tool-output, нет — пиши «не указано».
4. Dietary: ресторан без `dietary_confidence >= 0.5` не попадает в план.
5. Превышение бюджета — явный ⚠ маркер в конце дня.

Skill — это **контракт между LLM и UI**: гарантирует консистентность markdown во всех путях рендеринга.

RAG — Wikivoyage → Qdrant

```
Wikivoyage страница города
|
▼
chunking по секциям (See / Do / Eat / Drink / Sleep / Get around)
|
▼
OpenAI text-embedding-3-small (1536-dim)
|
▼
Qdrant collection trip_planner_v1
  metadata: { city, section, kind, source_url, ingested_at }
|
▼
city-knowledge.search_city_guide(city, query, section?)
```

5 indexed cities: Istanbul, Barcelona, Lisbon, Tokyo, Mexico City. Одна коллекция, фильтр по `city` — HE collection-per-city.

Мультимодальность — vision_identify

Опциональная нода, активируется когда пользователь загружает фото места:

1. Frontend: `PhotoUpload.tsx` → POST `/sessions/{id}/photo` (multipart, ≤ 5MB).
2. Бекенд: `base64` → `gpt-4.1-mini` (vision-режим, тот же mini что и для plan-генерации).
3. Output: `PhotoAnalysis { landmark, city, place_type, description, confidence }`.
4. Если `confidence >= 0.6` — landmark добавляется как обязательный пункт плана через `enrich_input`.

Демо: фото Голубой Мечети → `{landmark: "Blue Mosque", city: "Istanbul", confidence: 0.95}`.

Evals — dataset + метрики

Golden dataset: `trip-planner-golden-v1` в LangSmith, **10 примеров**.

Покрытие: 5 indexed cities, 1 non-indexed (Paris, для graceful fallback), 2 photo-кейса, 4 dietary (halal, vegan, vegetarian, gluten-free), 3 edge (low budget, contradictory interests, no-index city).

2 LLM-as-judge метрики (gpt-4.1-mini, temperature 0.0):

Метрика	Что проверяет	Target
<code>constraint_adherence</code>	budget $\leq$ заявленного, dietary соблюдены, дней ровно столько	≥ 0.80
<code>faithfulness</code>	каждое место в плане заземлено в tool-output	≥ 0.95

Автопрогон через `langsmith.evaluate` с sync-обёртками `async-judges`.

A/B результаты

`gpt-4.1-mini` (primary) vs `gpt-4o-mini` (secondary). Только модель меняется; embedding, RAG, prompts, MCP-tools — идентичны.

Metric	gpt-4.1-mini	gpt-4o-mini	Δ
<code>constraint_adherence</code>	0.800	0.800	0.000
<code>faithfulness</code>	0.965	0.507	-0.458

Вывод: `gpt-4o-mini` галлюцинирует ~половину названий мест. На `constraint_adherence` паритет, но `faithfulness` — drama. Hard-правило «места только из RAG/tool-output» делает её непригодной для primary.

Артефакты: `evals/results/mini-41.csv`, `mini-4o.csv`, `ab_mini_41_vs_4o.md`. LangSmith projects: `mini-41-8980574a`, `mini-4o-ebf8458a`.

LLM choice rationale

Выбор: OpenAI mini family. Primary — `gpt-4.1-mini` (везде: vision, generate_plan, parse_edit_intent, judges, finalize).

Критерий	<code>gpt-4.1-mini</code>
Стоимость	~\$0.40/M input, ~\$1.60/M output
Tool-use / structured output	strict mode, with_structured_output — стабильно
Vision	встроено в ту же модель — одна нода <code>vision_identify</code>
LangChain интеграция	<code>langchain-openai</code> — самый зрелый wrapper
Длинный контекст	1M токенов (запас для stuff-context fallback)
Faithfulness (evals)	0.965 vs 0.507 у <code>gpt-4o-mini</code>

Не выбрали: Anthropic Claude (ограничение задачи), Google Gemini (пользователь сменил на OpenAI), Qwen (менее зрелая интеграция, vision отдельной моделью).

Hyperparameters + production patterns

Гиперпараметры:

- `temperature=0.7` для генерации плана (нужен креатив в описаниях; факты приходят из RAG)
- `temperature=0.1` для intent parsing (structured output)
- `temperature=0.0` для LLM-as-judge (воспроизводимость)
- `max_tokens=4096` для плана, `512` для intent
- `top_p=0.95`, ретраи через tenacity (3 попытки, base 1s)

Resilience patterns:

- Overpass: 5 попыток × 3 зеркала с backoff
- SSE: polling-fallback на `/state` каждые 3с, `X-Accel-Buffering: no` для nginx
- LangGraph: `AsyncSqliteSaver` checkpointing — можно возобновить с любого interrupt
- Бюджет HITL: флаг `budget_acknowledged` разрывает loop `explain_and_ask ↔ budget_check`

What I'd do next

Следующие шаги:

1. **Fallback на gpt-4.1 (full)** при низкой confidence — для сложных случаев когда mini галлюцинирует
2. **bge-reranker** поверх Qdrant — ожидаемый +5-10% к faithfulness
3. **Расширение RAG до 10-15 городов** + автоматический индекс новых из user-запросов
4. **Multi-day weather optimization** — если день дождливый, сместить outdoor activities на сухой
5. **Render deploy** — публичный URL для демо

Что осознанно НЕ делал:

- Auth / user accounts (sessions анонимные)
- CI/CD (учебный scope)
- Semantic cache (mini-модель дешёвая, не нужно)
- Voice / audio (out of scope)

Спасибо

Trip Planner — github.com/aidarfortraining/travel-agent

Demo: `docker compose up` → <http://localhost:5173>

LangSmith dashboard: `mini-41-8980574a` / `mini-40-ebf8458a`

Вопросы?